What problem are we solving, for whom, and at what moment?

Start from the problem, not from the wording

1. Start with the problem, not the wording

Human-centered design mindset

Begin with real user goals, constraints, and context; let features and copy follow from that.

System image and conceptual model

The product's UI and text are the only "story" the user sees; they must project a clear mental model of what the screen is for.

Mental models and mappings

Align behavior and labels with how users already expect things to work; mismatched expectations cause confusion.

Problem framing vs solution hunting

Define the real problem and user scenario before choosing components, flows, or phrasing.

Organizational and business constraints

Regulatory, performance, and support constraints are part of the design problem, not external noise.

What does the screen say before anyone reads a single word?

People see structure, not sentences

2. How people see: hierarchy, affordances, signifiers

Visual hierarchy and discoverability

People notice size, contrast, and position before they read; the primary action must visually stand out.

Affordances and signifiers

Affordances are what can be done; signifiers are visual cues that show where and how to act (buttons, underlines, icons).

Grouping and proximity

Items that are close, aligned, or boxed together are perceived as related; spacing and grouping define structure.

Mappings (visual side)

Layout should mirror relationships (filters above lists, actions near items); visual mapping teaches behavior.

Visceral first impression

Before any text is processed, users form a gut reaction: "This looks clear and safe" or "This looks messy and risky."

What will people actually read, skim, or ignore on this screen?

Design for scanning, not ideal reading

3. How people read (or don't)

01

Reading vs comprehension

Users can pass their eyes over text without absorbing it; comprehension requires attention and relevance.

02

Chunked information presentation

Break content into small chunks: one idea per sentence, short bullet lists, clear section headers.

03

Contextual framing

A short "why you're here" line at the top makes the rest of the screen easier to interpret.

04

Attention dynamics

Minds wander and habituate; long boilerplate and repeated warnings quickly become invisible.

05

Recognition over recall (terminology)

Consistent terms across screens let users recognize meaning instead of re-interpreting new phrases.

How much are we asking people to hold in their head at once?

Limit cognitive load; let the UI remember

4. Memory limits and cognitive load

Limited working memory (~4 chunks)

People can hold only a few pieces of new information at once; long, multi-condition screens overload them.

Knowledge in the world vs in the head

Offload key information into the interface (labels, summaries, previews) instead of relying on memory.

Recognition over recall (state)

Show previous choices and current state instead of referencing them abstractly.

Progressive disclosure

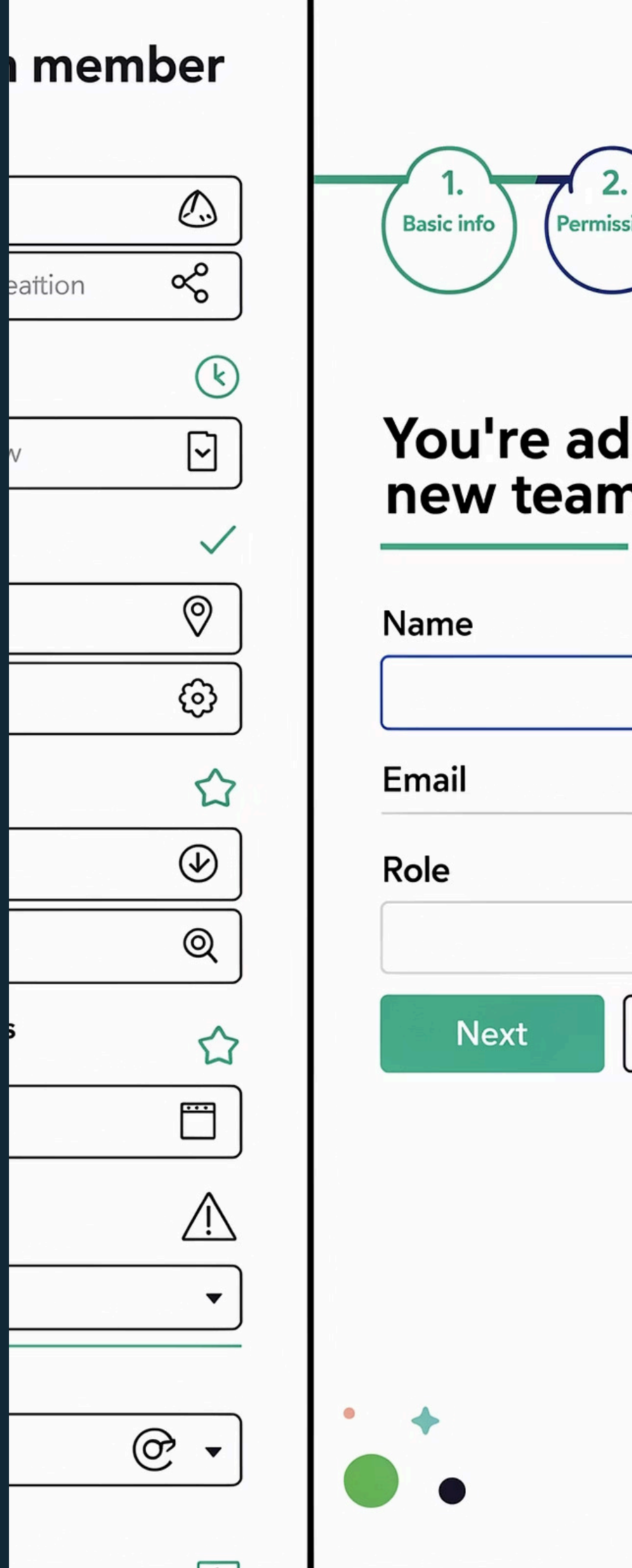
Reveal detail step by step behind "Show details", "Advanced", or multi-step flows rather than all at once.

Load trade-offs

Extra clicks and scroll are usually cheaper than asking users to remember complex conditions.

Re-entry and re-orientation

Every step should restate what task you're in so users can recover after interruptions.



What decision are we asking for here, and how risky or effortful does it feel?

Decisions are emotional, not just logical

5. How people decide, feel, and stay motivated

1

Decision load and choice architecture

Too many or too-similar options stall decisions; grouping and smart defaults reduce friction.

2

Emotion in decision-making

Perceived safety, trust, and fear of loss heavily influence which option feels acceptable.

3

Goal gradient and progress

Visible progress ("Step 2 of 3") makes people more likely to continue and finish.

4

Narrative persuasion

Short, concrete examples clarify choices better than abstract descriptions.

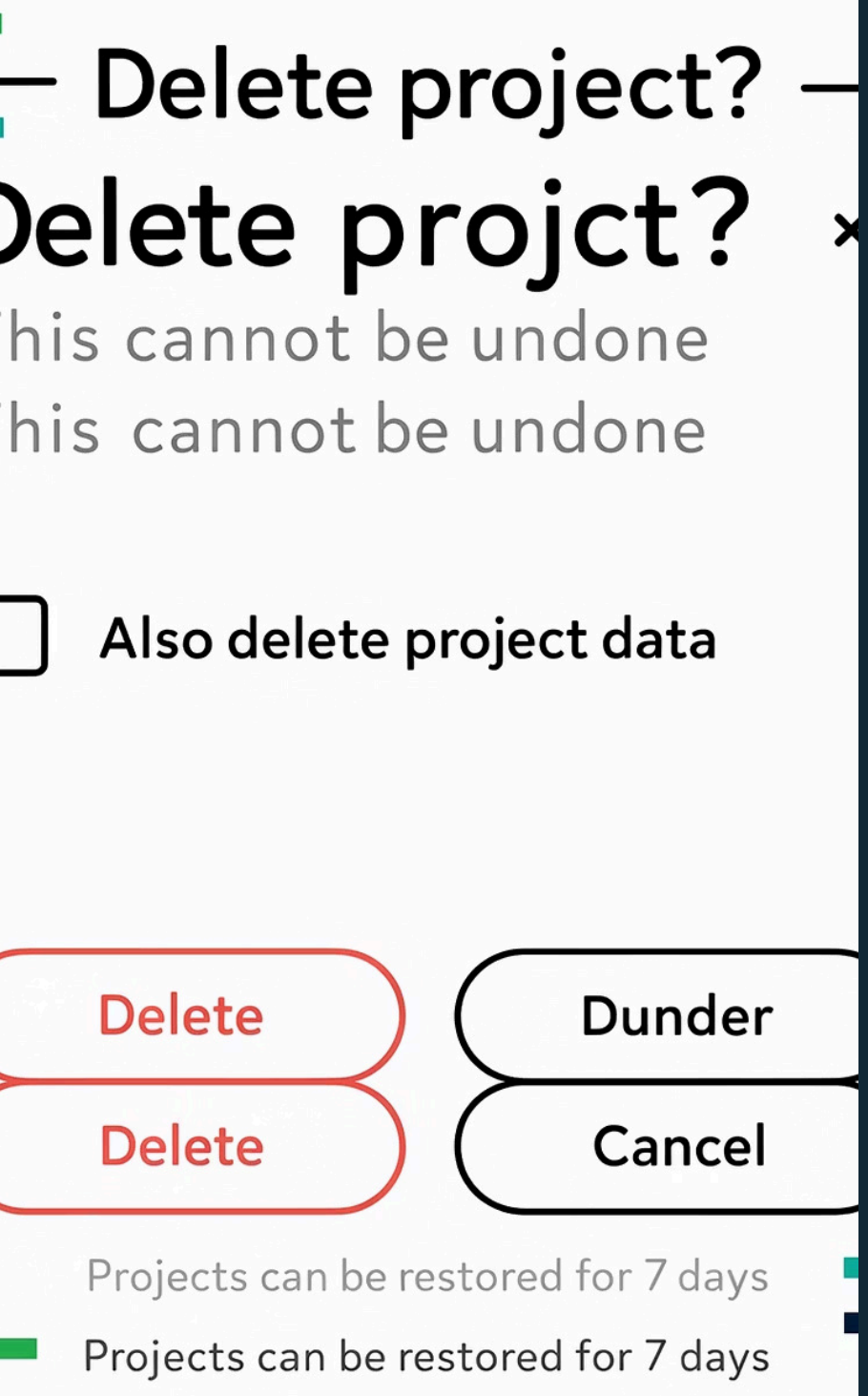
5

Time and effort framing

Stating effort ("Takes about 2 minutes") helps users commit more than just saying "quick".

If someone makes a mistake here, what happens to them?

Errors are inevitable; design the safety net



6. How people make mistakes (and how we absorb them)

1

Slips vs mistakes

Slips are execution errors with the right goal; mistakes are planning errors with the wrong goal or rule.

2

Constraint taxonomy

Physical, logical, semantic, and cultural constraints can prevent many errors before they happen.

3

Forcing functions

Interlocks, lock-ins, and lock-outs enforce order or preconditions so dangerous actions cannot happen accidentally.

Design for error

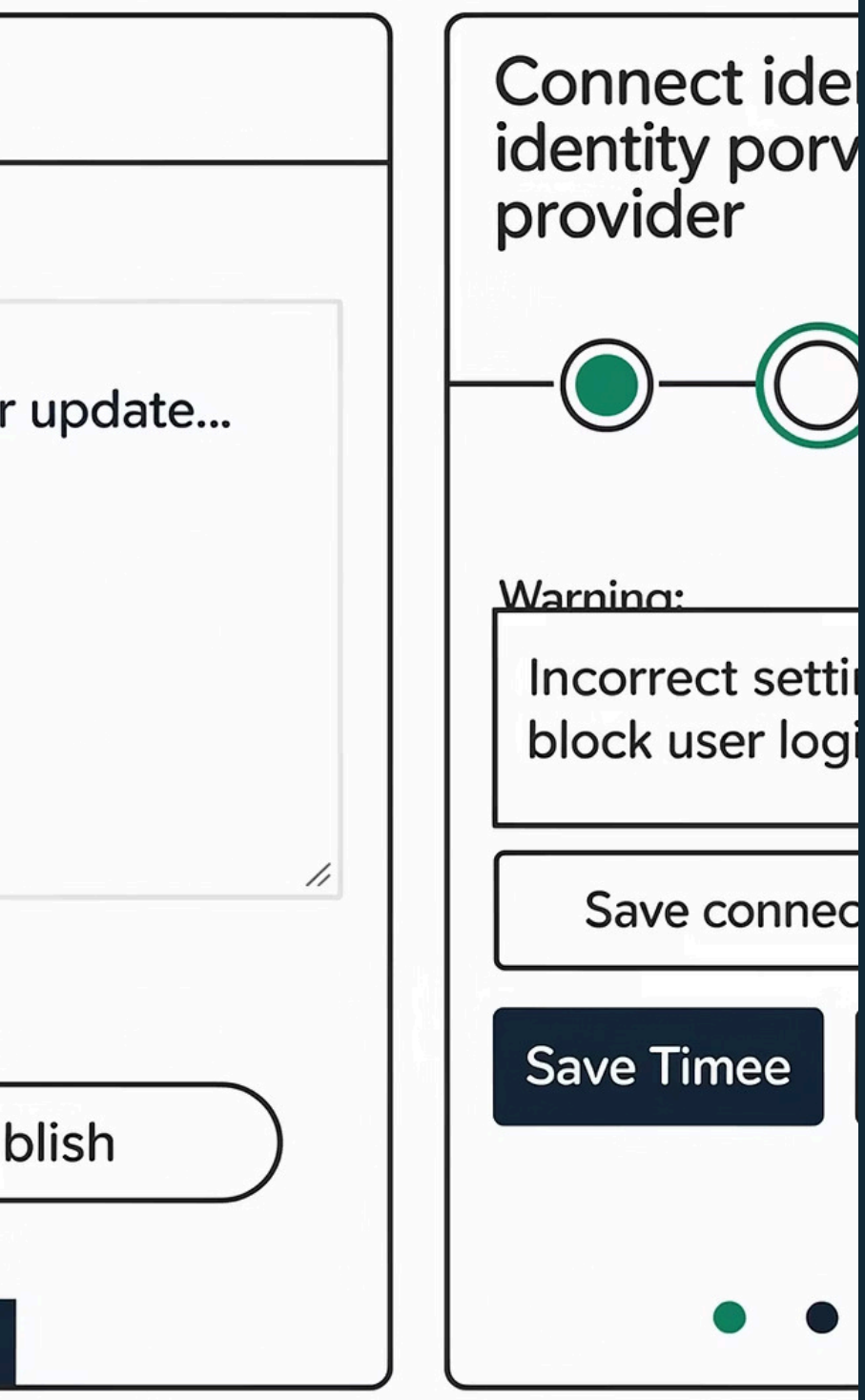
Build in undo, previews, soft deletes, and clear recovery paths; treat them as first-class design.

Gulfs of execution and evaluation

Many "errors" occur because users cannot see what to do or cannot interpret what just happened.

What kind of task is this: everyday routine, or scary one-time setup?

Frequency and risk should shape the flow



7. Tasks, flows, and UI surfaces

1

Task frequency and fluency

Frequent, low-risk tasks should be streamlined and fast; rare, high-risk tasks should be slower and more guided.

2

Norman's action cycle

Support the chain: goal → intention → action → perception → interpretation → evaluation.

3

Gulfs of execution and evaluation (flow level)

Each step needs a visible next action and feedback that makes the result understandable.

4

Habit formation and fluency

Repeated flows can rely on muscle memory and minimal explanation; one-off flows cannot.

Re-entry and re-orientation

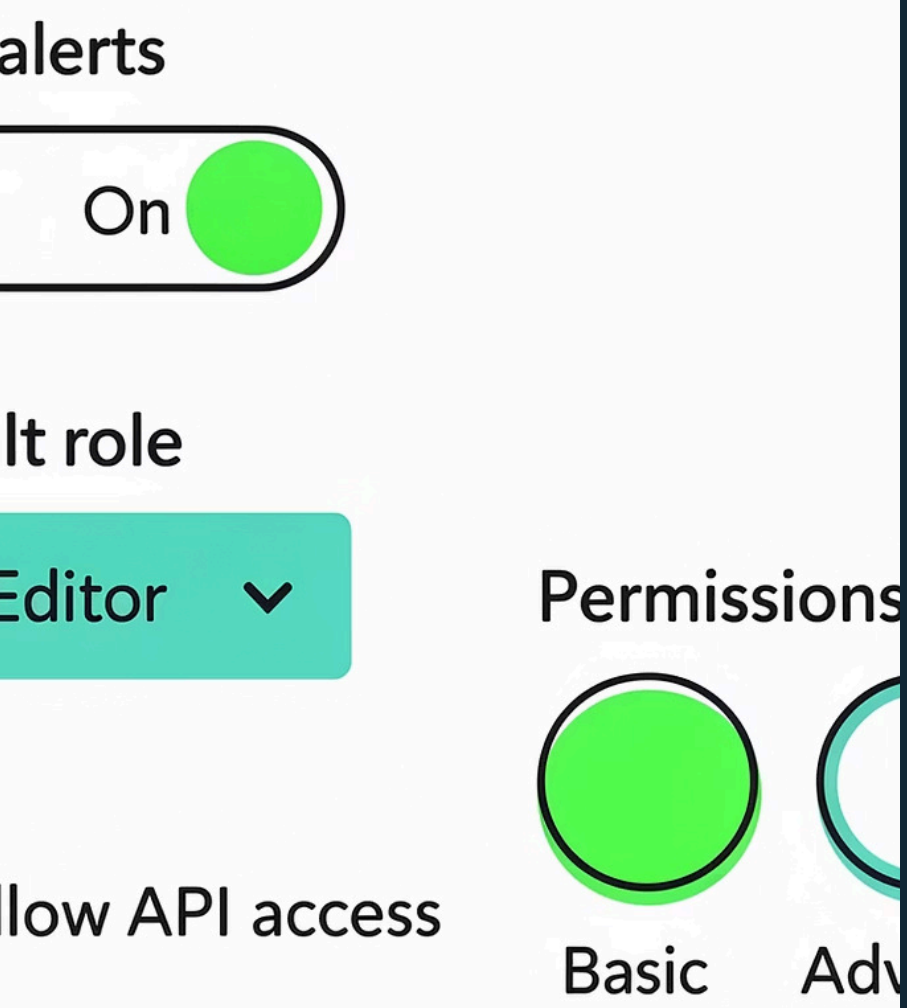
Longer flows need step names, progress indicators, and clear review points.

Progressive disclosure in flows

Use wizards, steppers, and "Advanced" sections to stage complexity across steps.

Can we let the design system do more work so the text can do less?

Use components as part of the language



8. Use the design system so content doesn't carry everything

Affordances and signifiers (component grammar)

Component shapes (buttons, toggles, links, toasts) already imply behavior; choose them deliberately.

Constraints via components

Radios enforce single choice, checkboxes allow multiple; disabled states block invalid actions instead of just warning about them.

Mappings in layout

Place filters near lists, actions near selected items, and errors next to failing fields so relationships are obvious.

Knowledge in the world via tokens

Status badges, icons, and color tokens encode meaning visually; copy should not duplicate them.

Conventions and standards

Reuse established patterns for confirmation, warnings, navigation, and feedback instead of inventing new ones.

Given all this, what is the job of the words on this surface?

Text is part of the system, not decoration

9. Writing inside the system (microcopy → UX writing → content design)

System image and mental model

Every label, message, and empty state contributes to how users think the system works.

Microcopy editing (T1)

Improve clarity, correctness, consistency, and tone inside an existing design.

State-aware UX writing (T2)

Write from scratch based on user state, task frequency, and risk; use patterns like "what / why / what happens next / how to undo".

Content design (T3)

Decide what information belongs on which surface (screen, dialog, toast, email) and push back when structure fights understanding.

Narrative examples and social cues

Short examples and light social proof clarify intended use better than abstract labels alone.

How do we explain and test our design decisions so they survive meetings?

Win with rationale, not volume

10. Explaining, testing, and evolving decisions

Articulated design rationale

Ground arguments in user goals, cognitive load, task frequency, and error risk, not personal taste.

Feedback loops

Use usability tests, metrics, and support tickets to refine language and flows over time.

Design thinking cycle

Treat copy and flows as hypotheses: observe → reframe → propose → test → iterate.

Decision load and memory constraints as arguments

Use concepts like "cognitive load", "recognition over recall", and "error tolerance" explicitly in design discussions.

Human-centered vs politics-centered

Anchor disagreements in user impact and outcomes rather than hierarchy or preferences.